

AEGIS Data Flow and Telemetry Guide

1. Physical Execution Topology

To establish a successful connection, the network boundary between the Windows host and the WSL2 virtual machine must be bridged, as they do not share the same localhost by default.

Component	Environment	Network Context	Role
KSP Application	Windows OS	Host Machine	Runs the physics engine and game state.
kRPC Server	KSP Mod (Windows)	Default Ports: 50000 (RPC), 50001 (Stream)	Exposes KSP data and control hooks via TCP.
Python Agent	Arch Linux (WSL2)	Hyper-V Virtual Network Adapter	Executes the control loop and Kalman filter.

Connection Resolution Paths:

- **Path A:** Enable `localhostforwarding=true` in `.wslconfig`.
- **Path B:** Target the Windows Host IP (found in `/etc/resolv.conf` within WSL2).
- **Path C:** Configure kRPC to bind to "Any IP Address" instead of strictly localhost.

2. Data Flow & Control Loop (Tick-by-Tick)

The system operates on a tight 20ms time budget per KSP physics tick (50Hz). It utilizes dual channels: a **Stream Channel** for automated, high-frequency telemetry pushes, and an **RPC Channel** for imperative, low-frequency control writes.

The Control Loop Sequence

Phase 1: Ingestion (KSP → Python)

↓ *KSP Physics Tick Triggers (20ms budget begins)*

↓ **Stream Channel** pushes `noisy_alt`

↓ **Stream Channel** pushes `raw_accel`

↓ **Stream Channel** pushes `vessel.mass`

Phase 2: Processing (Python `run_loop`)

↓ Inject Gaussian noise into altitude and acceleration data

↓ **Kalman Filter:** `estimator.predict(noisy_accel, dt)`

↓ **Kalman Filter:** `estimator.update(noisy_alt)`

↓ Retrieve refined state estimate

↓ **Fault Detection (FDI):** Check measured vs. expected acceleration

↓ *Evaluate:* Isolate fault if detected

↓ **Guidance:** Calculate `desired_wrench` based on state

↓ **Allocator:** Build B matrix, check condition threshold, and output throttles/gimbals

Phase 3: Execution (Python → KSP)

↓ **RPC Channel:** Set engine throttles (~1ms round trip)

↓ **RPC Channel:** Set gimbal angles (~1ms round trip)

↓ **Mission Director:** Evaluate and execute state transitions

■ *Tick Complete (Elapsed Python time: ~3–5ms / 20ms budget)*

3. Telemetry & Debug Surface

Because pausing the Python execution disrupts the real-time `dt` calculations of the Kalman filter, traditional breakpoints are replaced by high-resolution buffered logging.

File / Directory	Type	Cadence	Primary Purpose
<code>logs/latest/</code>	Symlink	Constant	Quick access to the most recent run data.
<code>telemetry.csv</code>	CSV	50Hz (Per Tick)	High-resolution quantitative data (altitude, acceleration, state matrices).
<code>events.jsonl</code>	JSON Lines	Event-Driven	Discrete milestones, faults, and system anomalies.
<code>run_config.json</code>	JSON	Once per run	Static initial conditions, seeds, and thresholds.

4. Event Timeline Graph (Conceptual)

The `events.jsonl` file generates a discrete timeline of the mission. When an anomaly occurs, the agent uses these timestamps to isolate the exact moment in the high-resolution `telemetry.csv` to investigate.

Time (s)	Event Type	Mission State	Trigger / Metric	Action Taken
312.4	STATE_TRANSITION	POWERED_DESCENT → TERMINAL_DESCENT	Altitude crossed 48.2m	Engaged terminal guidance protocols.
319.1	GREMLIN_KILL	TERMINAL_DESCENT	Engine Index: 2 failed	Simulated fault injected.
319.2	FAULT_DETECTED	TERMINAL_DESCENT	Accel Dev: 4.31 > 0.5 threshold	Fault detected 0.1s after failure.
...	DT_SPIKE	Any	$dt > 3 * expected_dt$ (Game Paused)	Skipped Kalman predict step to prevent filter divergence.